

CS3217: Software Engineering on Modern Application Platforms

AY 2023/2024, Semester 2

Sprint Report 1

TowerForge

Name	Matric Number
Keith Gan	[redacted]
Phua Zheng Ze	[redacted]
Rubesh Suresh	[redacted]
Vanessa Mae	[redacted]

Table of Contents

1 – Project Overview....4
2 – Features and Specifications....5
3 – User Manual....6
4 – TowerForge Design Overview....8
Top Level Design....8
Entity Component System....8
Events API....10
Rendering API....11
Input API....11
Usage of Libraries....12
Designs Problems....12
Rendering API....12
Events API....12
Movement System....13
5 – Runtime Structure....14
6 – Module Structure....16
7 – Reflection....18
Lessons....18
Evaluation....18
Overall Reflection....19
Known Bugs and Unresolved Issues....19
8 – Appendix....21
A – GUI....21
B – Test Cases....22
C – Project Schedule....23
Sprint Objectives....23
D – Individual roles and contribution....24

CS3217 Sprint Report I: TowerForge



1 – Project Overview

Preface

TowerForge is a strategic tower defense game set in a fictional world where players must defend their fortress from waves of enemy invaders. Players strategically place various towers along the enemy's path to destroy them before they breach the fortress gates and lose. As the game progresses, players earn game points to place more fortresses or summon more units and unlock to fend off increasingly challenging waves of enemies.

Overview

We plan to develop a grid based tower defense builder. Players should be able to create their own levels with a level builder as well as specify the rules that govern the game world such as positions where towers can be placed and the like. We hope to allow players to be able to make relatively simple customisations within the application and provide an easy to extend source code for players to further customize the game to their vision.

Basic Application Usage

In the level designer, players can specify parameters such as path arrangements, enemy types, enemy counts, minion timer and victory conditions, etc. Players will be given the ability to design the specific rules according to which the game will be played. This way, the players will have maximum control over the exact kind of game experience that they want to have.

Gameplay

During the game itself, players can place units (minions) to **attack**. Players can decide where they want minions in order to attack the opponent player and hence reduce their Health Points (HP). The player whose tower HP reaches zero first will lose the game.

Scope of Sprint 1

While our final goal for TowerForge is to allow players to customize their own levels and gameplay, for Sprint 1, we have constrained the requirements to build and implement the gameplay aspect first, without the Level Designer. This would allow us to build a solid and functional foundation with a seamless gameplay experience, upon which we can add more features in the later stages of the project, such as the Level Designer and Multiplayer.

2 – Features and Specifications

MeleeUnit	- A movable unit that deals damage on contact with units of opposing team - Will move towards the enemy base. Stops when there is an enemy entity in contact. - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
SoldierUnit	- A movable unit that is able to spawn Bullets to deal damage to entities of the opposing team. - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
ArrowTower	- An immovable unit that can be placed on the map to protect a lane. - Able to spawn Bullets to to deal damage to entities of the opposing team - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
Bullet	- A movable projectile that is able to deal damage with entities of the opposing team. - Disappears when dealing damage to an entity of the opposing team
Team	- Represents the player. - Contains the number of lives left by the player. This represents the number of entities that needs to be let into the base for the player to lose - Contains and generates the points that are used to purchase units.
AiComponent	- Acts as the player 2 in single player mode. - Randomly chooses a random unit and spawns it on a random lane on the board provided that it has enough points to spawn the unit. - A random delay is added before it attempts to spawn another unit.

3 – User Manual

Welcome to TowerForge!

Objective
TowerForge is a tower defense game. Your objective is to build an army that can defeat your opponents before the time runs out. In this game you have three base units: Melee, Soldier, and Arrow Tower.

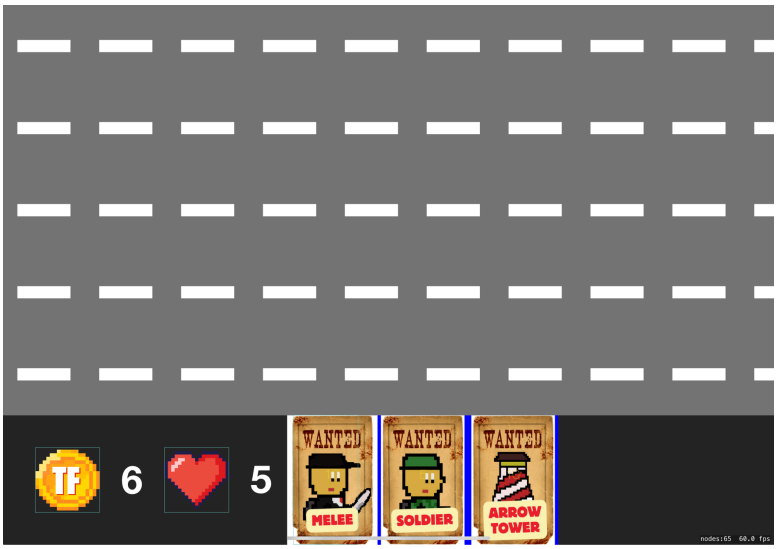
Combat rules
A Melee unit will only begin attacking when it contacts an enemy unit. A Soldier unit will begin firing as soon as an enemy unit is within its firing range. Lastly, an arrow tower is a static defense unit which is able to block enemy units while firing projectiles at incoming enemies.

Victory Criteria
When your units cross into the enemy's end of the screen, the enemy's health is depleted and vice versa. Your objective is to strategically place your army's units to defeat your enemy before the time runs out.

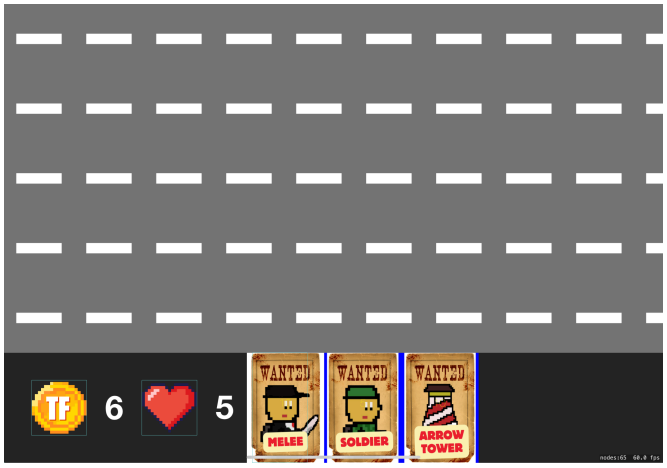
Demo
Shown below is the starting screen which will be shown on app launch.

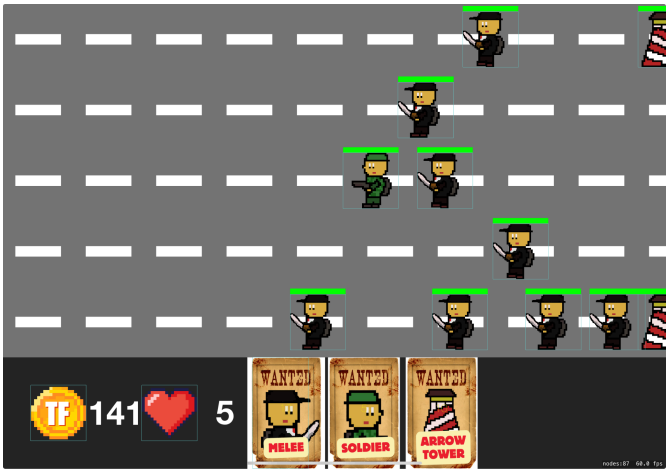


Clicking the play game button will bring you to the game scene.



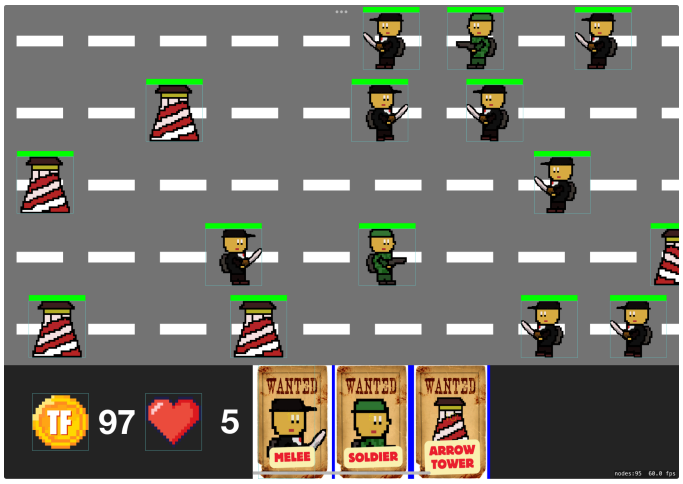
As time goes by, the number of coins you have will increase. You can use these coins to purchase units. Additionally, as time goes by, enemy units will spawn on the right side of the screen and move towards the left side of the screen.

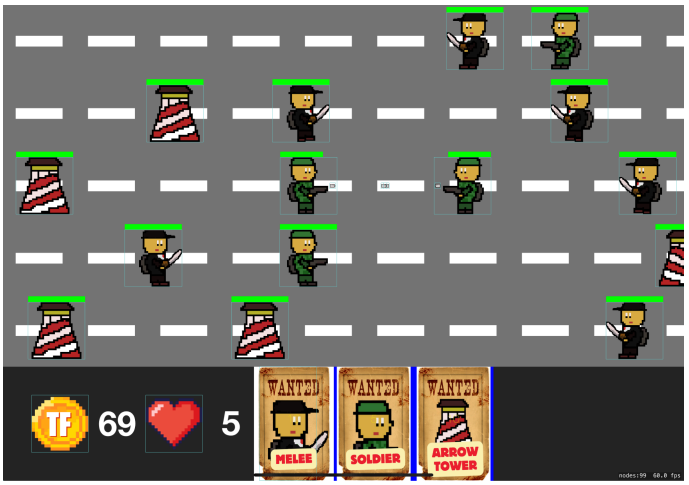




The left side of the screen indicates the user's "base" and the right side of the screen represents the enemy's "base". Enemy units will move towards the user's base and vice versa.

With coins, the user can generate more units and place them on the grid.





As shown on the right, as units come within firing range of each other, they will fire at each other, causing their health points to go down according to the damage dealt. After a certain number of hits, the units will "die" which will be represented by their disappearing from the game board.

Units that survive will continue to march towards their respective opponent bases.

When a unit passes the left edge of the screen, the user's life points will decrease. This is denoted by the number beside the heart in the selection bar at the bottom. When this life point reaches 0, the user will have lost the game.

The same applies to the enemy. Thus, the goal of the game is to make the opponent's life points reach 0. The first one to make the opponent's life points reach 0 wins the game.*

**The victory condition is not yet fully implemented, this is noted in the known issues part in this report*

4 – TowerForge Design Overview

Top Level Design

TowerForge consists of 4 main APIs. These include the Entity Component System(ECS), the Events API, the Rendering API and Input API. They are shown in Figure 1 below.

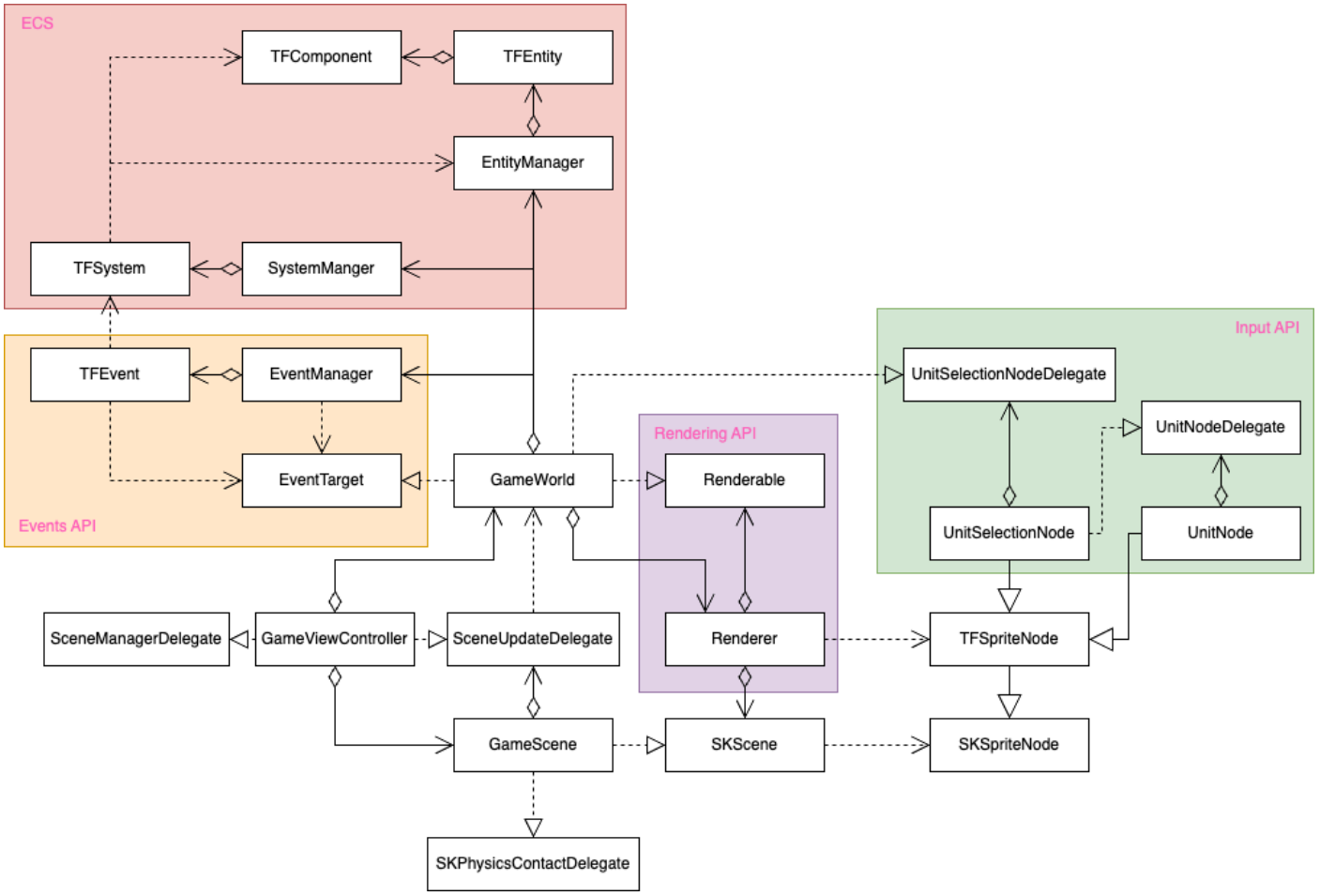


Figure 1: Top Level Class Diagram

Entity Component System

TowerForge is developed upon a robust and well-known game architectural foundation that leverages the **Entity-Component-System (ECS)** paradigm to manage the game entities, behaviors and interactions. **TFEntity** represents the game entities and the fundamental building blocks of the game world / level. **TFComponent** encapsulates the data and behaviors while **TFSystem** orchestrates the processing of entities with specific component compositions so they are more like the features of the game object.

This **ECS** framework promotes code reusability and enables simpler addition and removal of features and content. Figure 2 shows a high-level overview of the **ECS** framework applied to our TowerForge game.

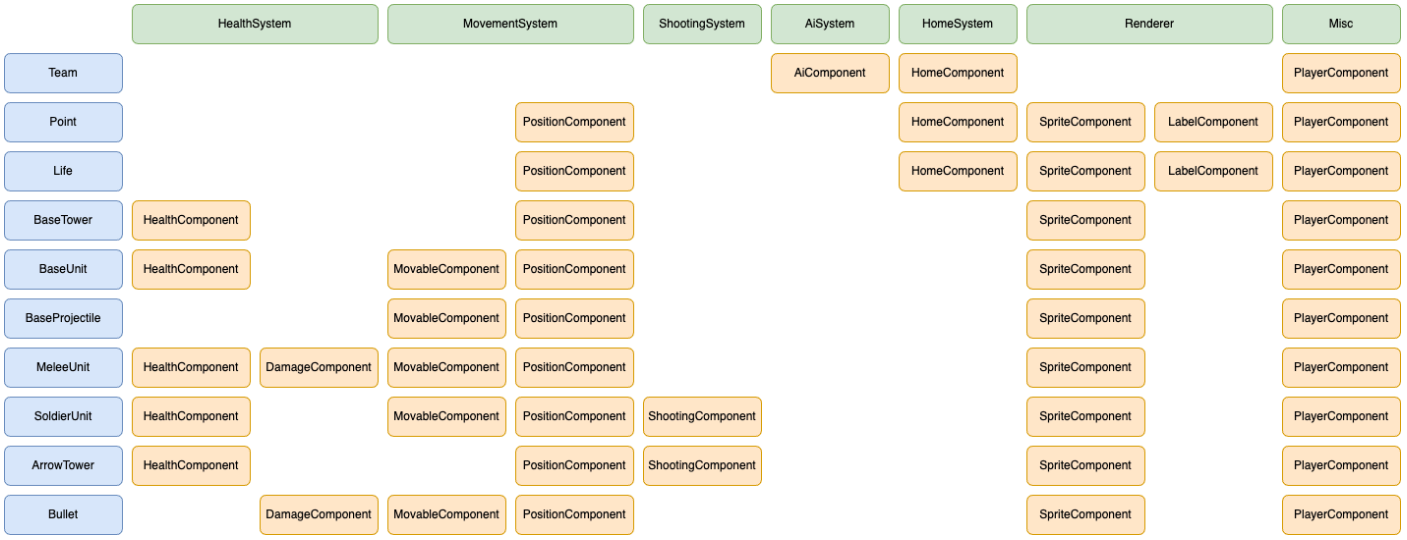


Figure 2: Overview of ECS Framework

The **ECS** framework is supported by the **EntityManager** (Figure 3) which holds all the entities that exist in the GameWorld at any point in time. Any **TFEntity** must be added to the **EntityManager** for it to be able to interact with other entities and for it to be able to be rendered. We can then access and remove the entity using its unique identifier. Each entity will also contain a list of **TFComponent**s which defines the behavior of the **TFEntity** that it is attached to. All **TFComponent** have a weak association with their owner **TFEntity** so that we can easily query the **EntityManager** to get the entity that has that specific component.

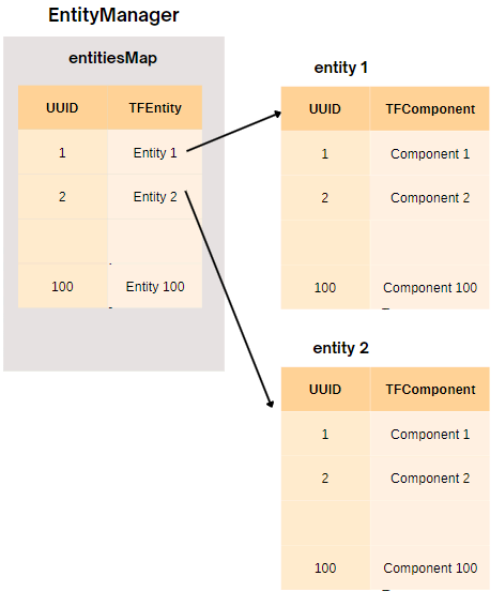


Figure 3: Overview of EntityManager

Each **TFEntity** and **TFComponent** are only to be modified by a **TFSystem**. This is to maintain the integrity of the data store in **TFEntity** and **TFComponent** which represents the game entity in the game world.

Events API

Events API serves as the means of communication between the different **TFSystem**s. Without this, . Each action that occurs in the **GameWorld** is represented by a **TFEvent**. Each **TFEvent** has a **TFEntity**'s id associated with it which represents the **TFEntity** that the event is occurring on. These events once created are placed into the **EventManager** where they are added into an **EventQueue** which executes them in FIFO (First In First Out) order. Once called to execute, these **TFEvent**s will call the appropriate **TFSystem**(s), via the **EventTarget** protocol, which contain methods for the **TFEvent** to execute the action that they represent.

Below is a run through of the generation and resolution of a **DamageEvent**.

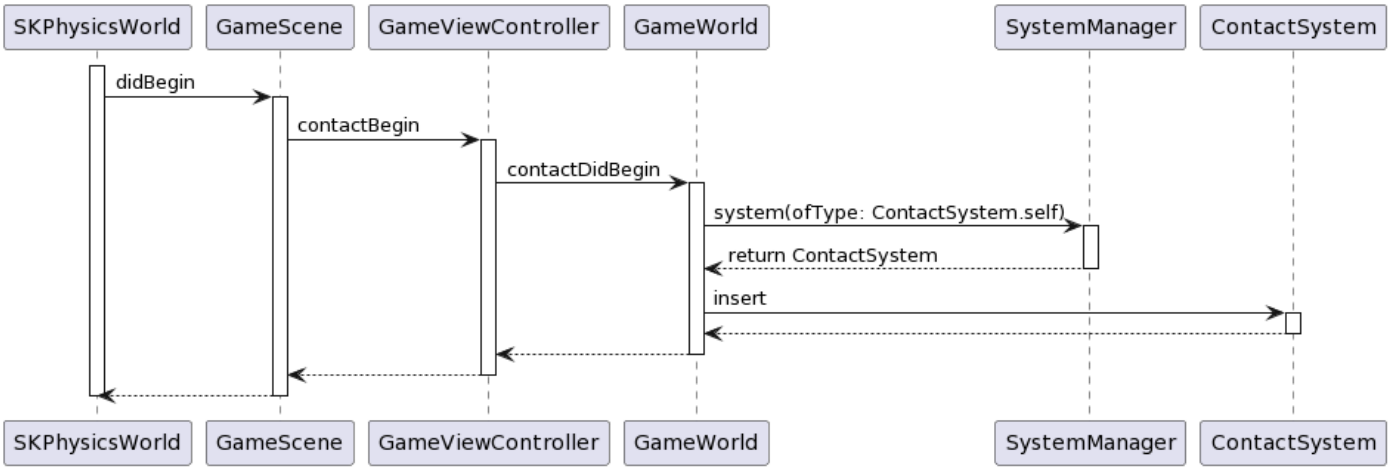


Figure 4: Sequence Diagram of TFContact Generation

When **TFEntities** that contain a **SpriteComponent** collide with each other, a **TFContact** representing the collision is generated and stored inside the **ContactSystem** until they stop colliding or one of the **TFEntity** is removed from the **GameWorld**.

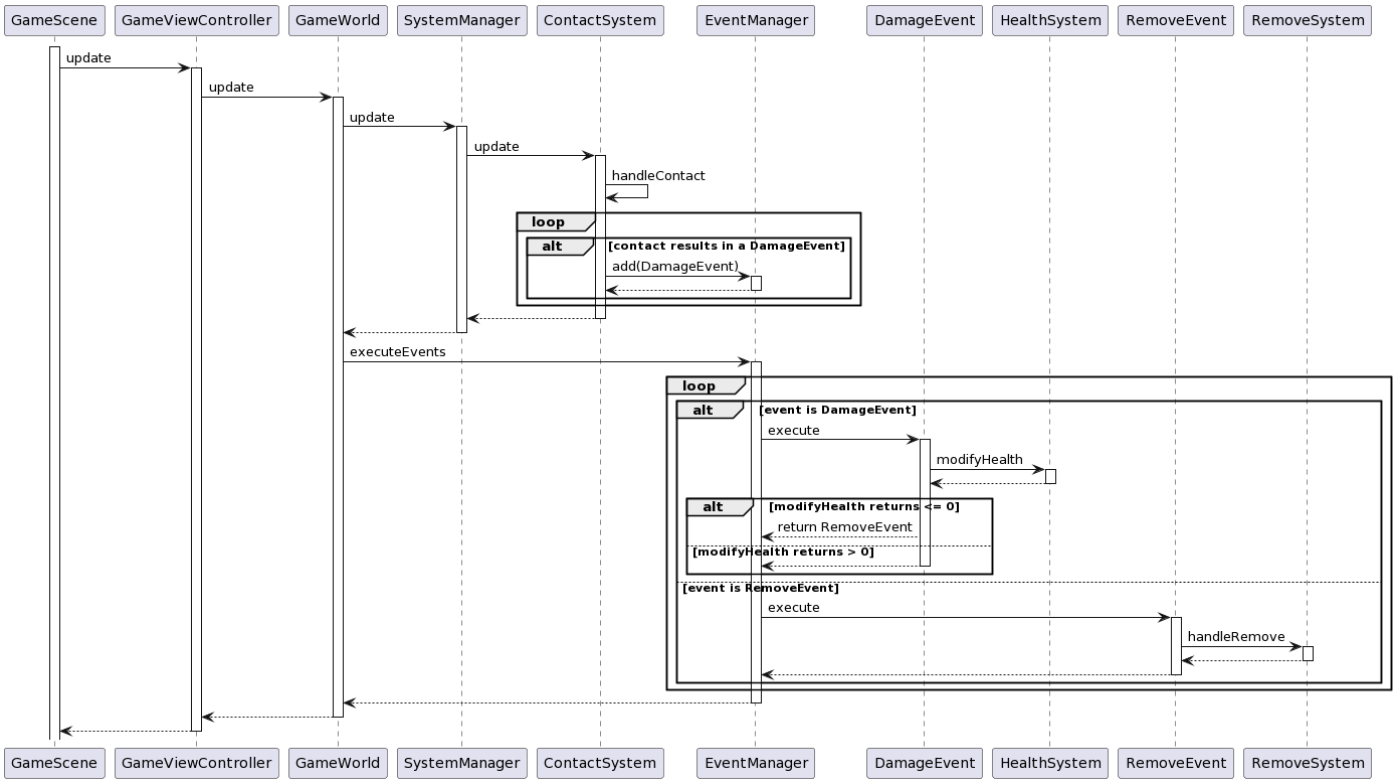


Figure 5: Sequence Diagram of Generation and Resolution of DamageEvent from TFContacts.

At every frame update called by the **GameScene**, the **GameWorld** calls for updates by all the **TFSystem**s that are attached to the **GameWorld.**

The **ContactSystem** iterates through all **TFContact**s and checks if they are still valid. After that the **TFContact** is evaluated which may result in a **TFEvent** which is then added to the event queue in the **EventManager** to await execution.After all **TFSystem**s have finished their updates, the EventManager then executes all the **TFEvent**s that have accumulated in the event queue.

If the **TFEvent** is a **DamageEvent**, it calls the **HealthSystem** to update the health of the damaged **TFEntity** that contains the id specified in the **TFEvent**. Once the health is updated, the **HealthSystem** returns the resulting health of the **TFEntity**. If the resulting health is less than or equal to 0, a **RemoveEvent** for the damaged **TFEntity** is returned by the **DamageEvent** to **EventManager** which adds the **RemoveEvent** to the event queue to await execution.

If the **TFEvent** is a **RemoveEvent**, it calls the **RemoveSystem** to remove the **TFEntity** with the id indicated in the **TFEvent** from the **GameWorld**. This would prevent other **TFEntities** from being able to interact with this **TFEntity** as well as being removed from the **Renderer** and will no longer be rendered.

Rendering API

The **Rendering API** is responsible for bridging the connection between the **TFEntities** that exist in the **GameWorld** and the **GameScene** which displays the **GameWorld** to the users. The **Rendering API** currently consists of just the **Renderer** and **Renderable** which is a protocol which exposes a method for **Renderer** to retrieve the **TFEntities** that they need to render.

The **Renderer** caches the **TFEntities** that have already been rendered and only updates them as needed. This reduces the cost of updating the view as compared to removing all existing view elements and rerendering all of them. The criteria for the action that the **Renderer** executes for each **TFEntity** is listed in Figure 6 below.

TFEntity	**TFSpriteNode**	**Description**	**Action**
Exists	Not cached	TFEntity not rendered	Add and Cache
Does Not Exist	Cached	TFEntity removed from world	Remove and Uncache
Exists	Cached	TFEntity rendered	Update

Figure 6: Criteria for Renderer Action

Input API

The **Input API** is responsible for mapping the user input into game actions. Currently, it serves to map user inputs into 2 different types of game actions: Unit Selection and Unit Spawn. Each **UnitNode** represents a **PlayerSpawnable TFEntity** by the user. Once selected by the player, it switches the current unit that will be spawned to be the **TFEntity** specified via the **UnitNodeDelegate**. Once the player taps on the level, it maps the location of the player input to the center game grid and spawns the **TFEntity** selected by **UnitNode** via the **UnitSelectionNodeDelgate**.

Usage of Libraries

We utilized **SpriteKit**'s components such as **SKSpriteNode**, **SKTexture**, **SKLabelNode**, and **SKActions** to create adaptors for our game engine. For example, we developed **TFSpriteNode** and **TAnimatableNode** by building upon **SKSpriteNode** and **SKAction**, respectively. These adaptors provided additional functionality and customization options tailored to our game's specific requirements. With **TFSpriteNode**, we extended the capabilities of **SKSpriteNode** to suit our game's rendering needs, while **TAnimatableNode** allowed for more sophisticated animations and visual effects to render the game units.

Designs Problems

Rendering API

Currently the **Renderer** does not create the **TFSpriteNode**s that are added to the **GameScene**. Instead, when the **SpriteComponent** is created, it creates the **TFSpriteNode.** Thus, when an entity is added to the **Renderer**, the **Renderer** takes the node that is stored in the **SpriteComponent** and caches a pointer before adding it to the scene to remember that it has been rendered.
However, this is probably going to change in the next sprint such that the **Renderer** will be the one creating the **TFSpriteNode**s for each entity so that there is proper separation of concerns.

Moreover, currently the **Renderer** assumes all **TFEntities** that need to be rendered will have a **PositionComponent** and a **PlayerComponent**. As such, UI **TFEntities** like **Life** and **Point** include a **PlayerComponent** even though it is not necessary for its behavior.
As such the Renderer will probably be changed to a pipeline where **TFSpriteNode**s are created for **TFEntities** with **SpriteComponent**. Subsequently, we can change the **TFSpriteNode** if they contain different components. Figure 7 below shows a sample rendering pipeline.

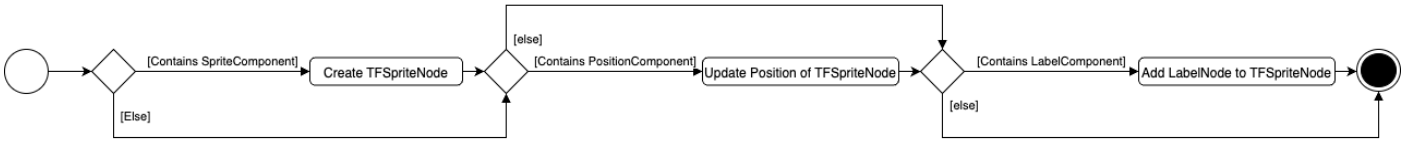


Figure 7: Activity Diagram Showing a Sample Rendering Pipeline

Events API

Currently in the EventManager, the event queue executes events in a FIFO order. This is fine but there are instances where certain events might possibly need to execute with a higher priority than other events. For example, assuming both **TFEvent**s are from the same **TFEntity**, a **RemoveEvent** should be executed before **DamageEvent**. This is because the **TFEntity** should already be removed according to **GameWorld** and thus should not be able to deal further damage to other **TFEntities** as it should no longer have any effect on the **GameWorld**. As a result we are probably going to change our event queue implementation from a FIFO queue to a priority queue.

Moreover, **TFEvent**s once added to the event queue are unable to be modified. This is not very adaptable as there are scenarios where once a **TFEvent** is executed, it might affect other **TFEvent**s that are waiting to be executed. For example, suppose TowerForge had powerups like a global damage boost for the team. Once executing the **TFEvent** that represents the damage boost powerup, all subsequent **DamageEvent**s that already exist in the event queue should be updated to reflect this new damage boost. However, there is no way to do that currently without the **TFEvent** representing the power up updating values in the various **TFSystem**s that the **DamageEvent**s will execute using.

As a result, we are probably going to change our event execution to have a pipeline similar to that proposed for the **Renderer** shown in Figure 7 above. With this pipeline **TFEvent**s are able to generate transforms for other **TFEvent**s that occur after it has executed. Thus when time to execute, a **TFEvent** will be updated with the relevant transforms before executing. Referencing the damage power up example above, once the **TFEvent** of the power up has executed, it can create a transform that multiplies the damage value in **DamageEvent**s, allowing us to be able to increase damage dealt easily.

Movement System

Currently, the **MovementSystem** updates the positions of entities by calling the update of **MovableComponent**s hence. **TFEntities** with a **MovableComponent** will always move in the direction of their velocities and there is no way for the **TFEntity** to stop moving under certain conditions. This is not very adaptable and there is little way to add custom behavior without having to hack a solution like the **ContactSystem** is doing currently to stop entity movement when 2 **TFEntity** of different **Team**s are colliding.
As a result, in the next sprint, we are probably going to change **MovementSystem** to use the **MoveEvent** that has been implemented. This would allow us to generate **MoveEvents** for each entity and to remove **MoveEvent**s for **TFEntities** that meet various criteria not to be moved.

5 – Runtime Structure

Relevant Classes

GameWorld
GameWorld is where the EntityManager, SystemManager, EventManager is initialized and set up. In the initialization of GameWorld, we set up the various systems present in the game along with the systems manager which is used to process systems, the event manager which is used to process events, and the entity manager which is used to add the initial entities in the game. For instance, we set up the teams and player information (points, life) on game initialization. We also add the various systems using the initialized system manager.

EntityManager
We use an EntityManager to manage the entities in the game. The EntityManager contains an entitiesMap which uniquely identifies a TFEntity with a UUID. The systems can then call on the entity manager to perform operations (retrieval, addition, deletion) on the components in the ECS module.

SystemManager
The SystemManager is used to manage the various systems present in the game. The SystemsManager is responsible for adding or deleting systems, which is done on game initialization. Then, its primary function during the game run is to update the various systems. The various systems are responsible for their own updates. For instance, the MovementSystem will handle the movement of the movable components in the different entities present in the game.

Implementation of CheckRepresentation

Overview
EntityManager and TFEntity both contain a dictionary where the key is the UUID of the element stored. EntityManager contains a dictionary called “entitiesMap” whose key is of the type UUID and the corresponding value is a TFEntity. Since each TFEntity contains a UUID as its unique identifier, this allows for fast retrieval of a given TFEntity, with O(1) average time complexity. It is possible to query a TFEntity without knowing any information about it. The same principle can be generalized to TFComponent as well.

Runtime Vulnerability
The universality of UUID and its inherent uniqueness (without requiring a central authority to validate its uniqueness) allows for multiple modules or systems that encapsulate TFEntity to exchange information about a specific TFEntity without directly exchanging copies of a TFEntity itself, by simply passing the UUID associated with that entity. However, this opens up the possibility of UUIDs being mutated and thereby introducing a vulnerability in the system. Even though the UUID of each TFEntity is a *let* constant, the keys can be mutated resulting in a situation where one the key doesn’t correspond to the id within the TFEntity that is the value corresponding to the key. The same principle can be generalized to TFComponent.

CheckRepresentation Function
To fix this vulnerability, a checkRepresentation function is introduced wherever there is a usage of a value whose id is the key of a dictionary data structure used elsewhere in the codebase.

```
74      private func checkRepresentation() -> Bool {
75          for (key, value) in components {
76              if key != components[key]?.id {
77                  return false
78              }
79          }
80          return true
81      }
82  }
```

Usage

This function can then be called to assert wherever that particular data structure is used. In our case, this corresponds to wherever TFEntity and TFComponent are contained, which are EntityManager and TFEntity respectively. Whenever this data structure is modified, the checkRepresentation function can be asserted. The private nature of the function ensures that the integrity of the data structure can be ascertained without ever having to expose that data structure to test modules.

TFEntity

```
33      func addComponent<T: TFComponent>(_ component: T) {
34          /// assert(checkRepresentation())
35          guard !hasComponent(ofType: type(of: component)) else {
36              return
37          }
38          component.didAddToEntity(self)
39          components[component.id] = (component)
40      }
41
42      func removeComponent<T: TFComponent>(ofType type: T.Type) {
43          /// assert(checkRepresentation())
44          guard let componentToBeRemoved = component(ofType: type.self) else {
45              return
46          }
47          componentToBeRemoved.willRemoveFromEntity()
48          components.removeValue(forKey: componentToBeRemoved.id)
49      }
```

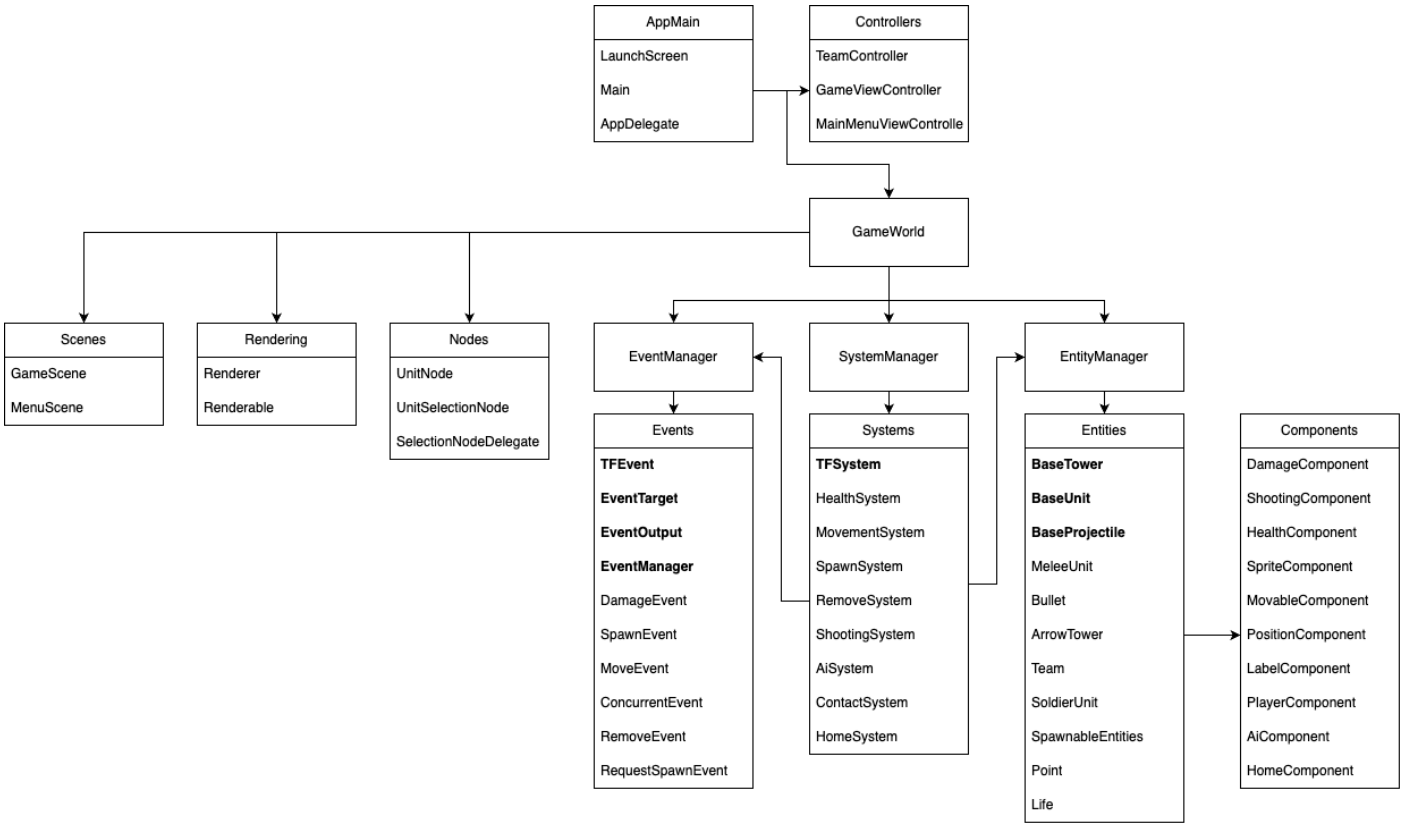
Entity Manager

```
25      func add(_ entity: TFEntity) {
26          entitiesMap[entity.id] = entity
27          /// assert(checkRepresentation())
28      }
29
30      func removeEntity(with id: UUID) {
31          entitiesMap.removeValue(forKey: id)
32          /// assert(checkRepresentation())
33      }
```

6 – Module Structure

Module Structure and Diagrams

Below is the module diagram for the game:



ECS Framework
TowerForge follows the ECS framework by separating the fundamental design into three parts : Entities, Components and Systems. More about this, including the design patterns used are explained in the Design overview section.

TFComponent represents a characteristic or trait that the game entity has. Each component can override the update(deltaTime:) method to make updates to the entities and components which work well with the **Game Scene**‘s update method.

TFSystem is where the component system lies and is responsible for handling updates. Each TFComponent will have an associated TFSystem.

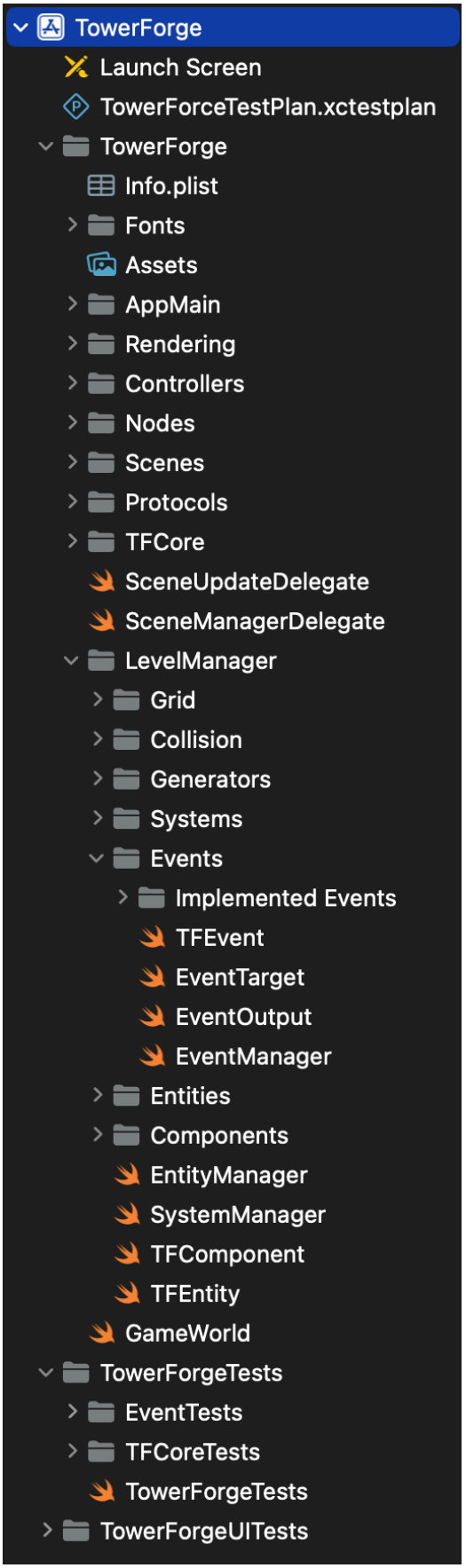
Unit Generator is used to spawn the units into the game. In this class, we have methods to spawn different units.

TFEvent represents an event that will be executed in the game. The game uses events to process changes in the game where each event corresponds to a single change in the game. For instance, the spawn event will spawn a unit in the game upon execution.

UnitSelectionNode is used to select the player’s unit that will be deployed in the game. The player can choose from the units that are available in the game and this will be reflected accordingly on the screen.

File Structure

We've organized the project as shown on the right. The TowerForge root folder contains the majority of the project file. Below are some files and groups explained (including those not included in the above module diagram).



TestPlan
The TowerForgeTestPlan file is the custom test plan we use, which allows us to have more control over the tests executed. This is particularly important because we use continuous integration to build and test our project using GitHub Actions, and having UITests run for every PR is very resource intensive and time intensive. Thus, we can use this custom TestPlan to disable UITests in the cloud while being able to run them locally when needed.

Fonts
This group contains the custom fonts that we use in TowerForge

AppMain
This group contains files related to the Application and App lifecycle, such as the AppDelegate class and storyboard files.

Protocols
This group contains protocols used universally across TowerForge, such as **Animatable** and **Spawnable**.

TFCore
This group contains base elements needed for GUI rendering, including some SpriteKit components.

LevelManager
This group contains files and classes responsible for the majority of game logic, including entities, components, systems, all the respective managers, and level-specific features such as the grid and unit generators.

It also contains the GameWorld class which is responsible for the majority of the Game Logic.

TowerForgeTests
This group contains unit tests and integrations tests.

TowerForgeUITests
This group contains UI tests.

7 – Reflection

Lessons

ECS is a very powerful design pattern for software that requires very customisable behavior. With **ECS**, in order to add new behavior to an entity, all that needs to be done is to create a new component to hold the data representation relating to the behavior and add a new system to update the components to reflect the expected behavior. This makes **ECS** very suitable for games where there are many different entities that might share various behaviors but have some differing behaviors. As such, it is hard to represent the relationship between the differing entities with just regular **Polymorphism**.

Since we are building a game that belongs to the Tower Defense genre, we decided to make use of the benefits of both **Polymorphism** and **ECS**. Since our game's entities can be clearly defined into 3 types with distinct behavior, we implemented the base class for each of these types. They are **BaseUnit**, **BaseTower** and **BaseProjectile**. **BaseTower** has a health and a position, **BaseUnit** is similar to **BaseTower** but it can also move and **BaseProjectile** does not have health but can move. By defining these base classes, we need not repeatedly write the same code for classes that clearly subclass these base classes while still being able to customize each of their behavior to the nature of **ECS**.

Evaluation

Working with Xcode

One thing that we had greatly underestimated was the issues that can arise from working on an Xcode project. In most other development environments, modified files are tracked directly and singularly, merged conflicts can be handled on a file-by-file basis. In Xcode however, new files are indexed within the .pbxproj file, where each file and the entire file hierarchy is represented in text, alongside the file itself within the project. This presented a major problem when we had to restructure the project's file structure, and that caused merge conflicts that could not be resolved at all. The merge conflict had been caused by the .pbxproj file which was modified due to changes in the folder structure. The corrupted .pbxproj file meant that the conflicting changes had to be manually added again. To avoid this issue in the future, we refrain from changing the project's file structure frequently and when we needed to do so, we ensure that no one had any changes on any existing branches.

ECS & OOP

Initially based on the project proposal period, we had planned to develop the game using a traditional **OOP** design, where entities were represented as classes with predefined properties and behaviors. The decision to adopt the Entity-Component-System (**ECS**) architecture in TowerForge stemmed from our group's recognition of its numerous advantages over a purely Object-Oriented Programming (**OOP**) approach.

By combining **OOP** and **ECS**, TowerForge achieved a balance between object-oriented design and data-driven architecture. **OOP** provided a familiar and intuitive way to define game entities and their behaviors, while ECS offered greater flexibility and scalability by separating data from behavior. This hybrid approach enabled developers to leverage the strengths of both paradigms, resulting in a well-structured, modular, and efficient game architecture.

The adoption of **ECS** required some adjustments to our development workflow and mindset, but ultimately, it proved to be a valuable decision. It allowed us to better manage entity behavior and interactions, reduce complexity, and facilitate the addition of new features with minimal disruption to existing code.

Overall Reflection

Tell me how you think you're doing so far. Are you able to meet your scheduled deadlines? Have you slipped? What did you do (or what do you plan to do) about the slippage?

We believe that we are doing quite well thus far, being able to keep up with our internal deadlines and our forecasted schedule for the project. There were some minor slippages here and there particularly when multiple people work on a single component or dependency, this would naturally introduce some bottlenecks as certain components have to be finished first before progress on the dependent component can be made. One example was that the AIComponent required Systems to be implemented first, which had not yet been implemented by the time AIComponent was completed. However, this also allowed us to rethink our design to be more compatible with SWE principles - we had to ensure that the components we implemented dependent as little on other parts of the codebase as possible. In doing so, not only were we able to make progress more effectively, we also learned to appreciate the importance and motivation behind some SWE principles by applying them first hand.

If you don't think you're doing well, what went wrong? What are the problems that your group is currently facing? How do you plan to deal with these problems?

As most of us are new to iOS development, we are also new to working with the Xcode integrated development environment. While we had worked with it individually during the individual project in the first half of the semester, this project is a collaborative effort and multiple people working on the same Xcode project presented its own issues (highlighted earlier). However, with proper communication, we were able to sort out all issues that we might have had, ensuring a seamless dev experience for all team members.

What have you learnt in the last two weeks? How are you going to apply that newfound knowledge to the rest of the project?

We had learned the importance of transparent communication and keeping to a schedule, ensuring that everyone is aware of what everyone else is working on and ensuring that we are all on the same page. This not only allows us to figure out any issues that might arise in the course of us working on TowerForge, but also ensures that everyone has a pleasant experience working together.

Known Bugs and Unresolved Issues

The current version of the game faces three notable issues.

Firstly, there's a need to adjust the game screen size to ensure optimal display across different devices. Currently, the game screen may not properly fit on all screen sizes, leading to visual inconsistencies and potential user experience issues. This will largely be mitigated once we implement the ability to pan about the **GameWorld**. With the ability to pan, we will be able to fix the size of the **GameWorld** and let the view adapt and instead of having to adapt the **GameWorld** to the size of the view.

Secondly, there's room for performance enhancement by implementing a mechanism to remove unnecessary nodes and entities that are no longer in gameplay. The absence of such a feature may result in unnecessary resource consumption and could degrade the overall performance of the game, particularly during extended gameplay sessions. Addressing these issues would contribute to a smoother and more immersive gaming experience for players.

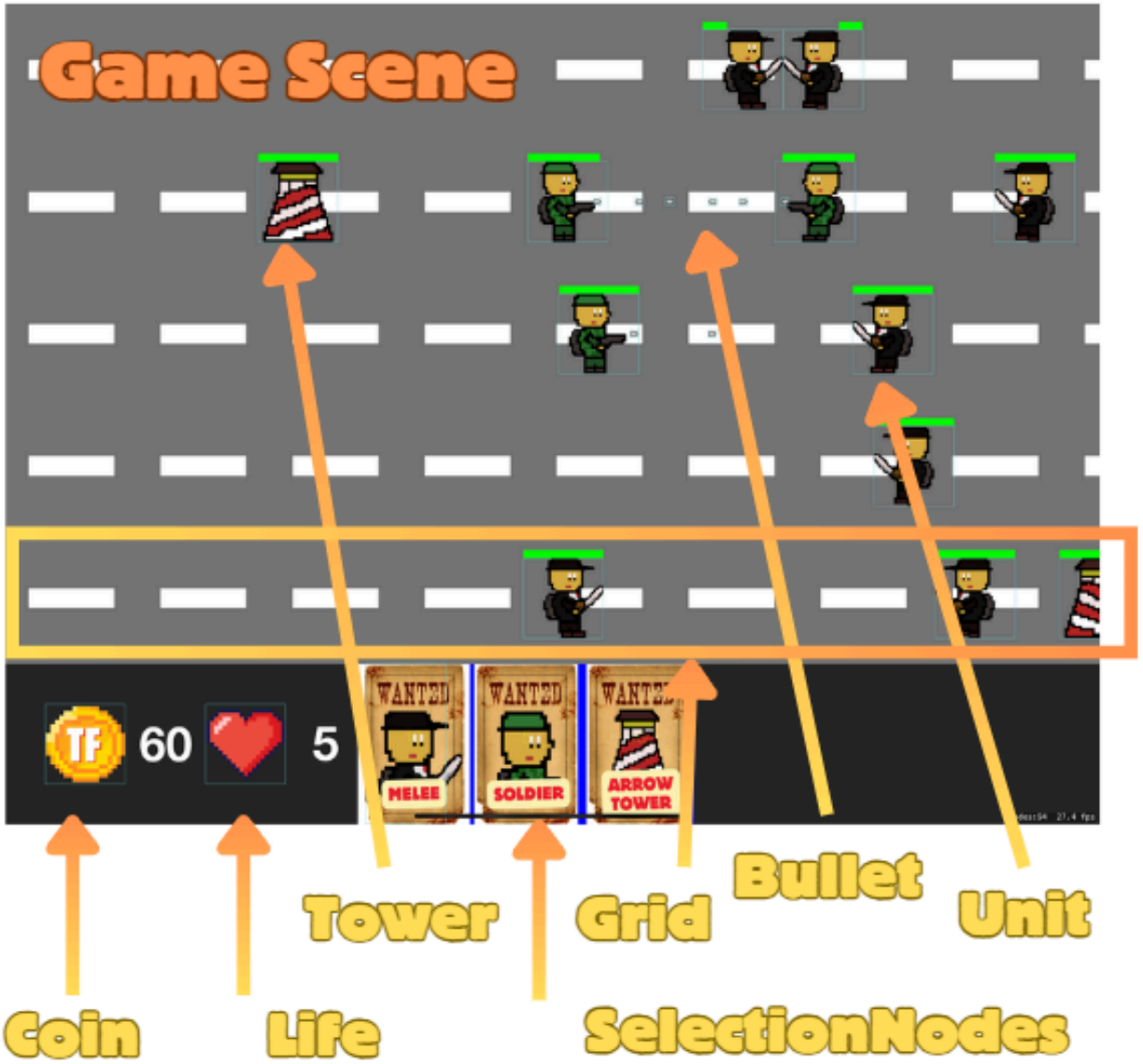
While there is a **RemoveSystem** which can be used to remove entities from the **GameWorld**, only the **HealthSystem** is making use of it to remove entities whose **HealthComponent**s health have reached 0 as

well as **TFEntities** that have a temporary **DamageComponent** to remove themselves after they have dealt their damage. As such, entities that do not meet any of the criterias above will never be removed from the **GameWorld** and will continue to consume compute resources.

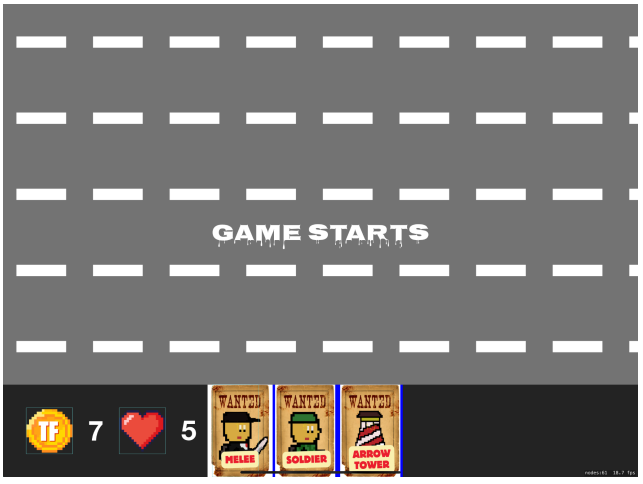
Thirdly, while we have implemented the **Grid** feature to normalize spawns to the center of the grid square, we have not implement the logic to specify spawning rules such as towers being able to be placed anywhere on the player's half of the board and units only being able to be spawned from the player's base.

8 – Appendix

A – GUI







B – Test Cases

Startup:

- Upon startup, the user should be redirected to the main screen where they can see a button “Play Game”
- Clicking on “Play Game” should redirect the user to the game screen

Game Screen:

- The user should see their available coins and life points on the bottom-left corner of the screen
- The user should be able to select between the different units (Melee, Soldier, Arrow Tower)

Spawning Units:

- With a unit selected, the user should be able to click on the screen to spawn the selected unit on a lane
- Upon spawning a unit, the available coins should decrease by the cost of the unit

Combat:

- All units should stop on contact with an enemy unit on the same lane
- Unit health points should decrease upon being attacked by an enemy unit
- When a unit’s health reaches zero, the unit should be removed from the game screen

Unit Behavior:

- Soldier units should begin shooting bullets when they are in range of an enemy unit on the same lane
- Melee units should attack on contact with an enemy unit
- Arrow Towers should block enemy units from advancing

C – Project Schedule

Sprint	Week	Date	Task
-	Week 7	March 3	Admin Tasks - Set up Workflow, CI/CD, etc
1	Week 8	March 10	Models (Grid, Game Entities, Persistence) + Unit test
	Week 9	March 17	Game Loop, Event Queue, Game Engine, UI + Controllers
2	Week 10	March 24	Behaviour Customisation (Types, Paths, Enemy Prioritisation)
	Week 11	March 31	Multiplayer Network Adapter* (or other extensions)
3	Week 12	April 7	Multiplayer Network Adapter* (or other extensions)
	Week 13	April 14	Finalize project and Bug Fixing

Sprint Objectives

Sprint 1 (March 10 - March 23)

- We want to have the basic features and have a robust skeleton of the game which will be easily extendable.
- We want to have rigorous unit testing for all the components that we implement.
- We want to have code coverage analysis and continuous integration workflows set up for the rest of the project.

Sprint 2 (March 24 - April 6)

- We want to have basic customisation and possibly implement one of the extensions.
- Developing and integrating into a working game.
- We want to begin integration testing for the features that we have implemented and set up a workflow to continuously test new features

Sprint 3 (April 7 – April 17 STePS)

- We want to include the bells and whistles + other extensions that would be viable and effective to be added to the game and also polish up the game for final distribution/release.
- Finalizing the game
- Closer to the deadline we would want to implement a feature freeze and work on perfecting the features that we already have, sorting out any possible issues.

D – Individual roles and contribution

Week 8 Todos

Task	Assignee	Status
TF Adaptors	Vanessa	Done
Components + TFComponents	Vanessa	Done
Entities + EntityManager + TFEntity	Zheng Ze	Done
Storage	TBC	Skipped
Grid class	Keith	Done
Level class	Keith	Skipped
Systems + TFSystem	Rubesh	Done
CI/CD + SwiftLint + Github setup	Rubesh	Done

Week 9 Todos

Task	Assignee	Status
Game Loop	TBC	Using SpriteKit Implementation
Game Engine	Rubesh	Done (Integrated with GameWorld)
Event Queue + Event Manager + Events	Zheng Ze	Done
GameScene + Game Controller	Vanessa	Done
UI + Level Designer Controller	TBC	Skipped
Enemy Spawner	Vanessa	Done
Collision	Zheng Ze	Done (Using SpriteKit's collision)
UI + Nodes	Vanessa	Done
Renderer	Zheng Ze	Done
GameWorld	Zheng Ze	Done

Week 10 Todos

Task (Behaviour Customization)	Assignee	Status
Types	TBC	
Paths	TBC	
Enemy Prioritization	TBC	
Spells	TBC	
Missions	TBC	

Game Modes	TBC	
Storage	TBC	
Power Ups	TBC	
Render Pipeline	TBC	
Event Pipeline	TBC	
Priority Queue	TBC	
Improve Movement and Contact System	TBC	

References

https://digitalcommons.calpoly.edu/cgi/viewcontent.cgi?article=1138&context=cpesp